

数据库安全性

数据库的安全性是指保护数据库以防止不合法使用所造成的数据泄露、更改或破坏。系统安全保护措施是否有效是数据库系统主要的性能指标之一。

对数据库的破坏来自以下：

- 非法用户
- 非法数据
- 各种故障或误操作
- 多用户的并发访问

采取的相应措施：

- 安全性保护（权限管理）
- 完整性保护
- 故障恢复
- 并发控制

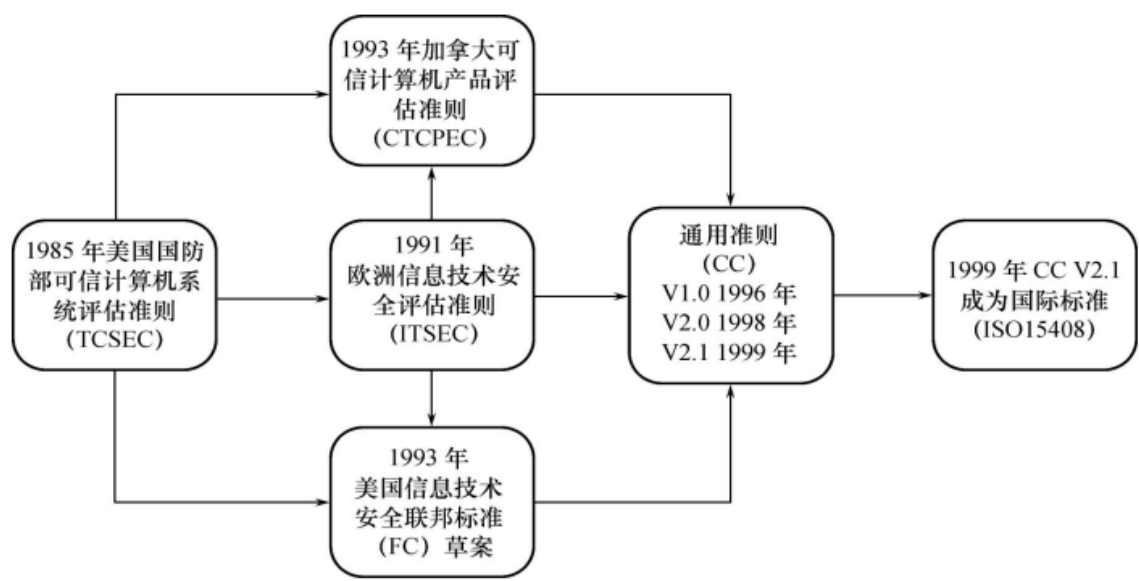
1.数据库安全性概述

1.1 数据库的不安全因素

- 非授权用户对数据库的恶意存取和破坏
- 重要信息或敏感数据泄露
- 安全环境的脆弱性

1.2 安全标准简介

图4.1 信息安全标准的发展历史



TCSEC/TDI安全级别划分：

按系统可靠或可信程度逐渐增高

各安全级别之间向下兼容

安全级别	定 义
A1	验证设计（Verified Design）
B3	安全域（Security Domains）
B2	结构化保护（Structural Protection）
B1	标记安全保护（Labeled Security Protection）
C2	受控的存取保护（Controlled Access Protection）
C1	自主安全保护（Discretionary Security Protection）
D	最小保护（Minimal Protection）

CC:提出国际公认的信息技术安全性的结构

CC文本组成：

- 简介和一般模型
- 安全功能要求
- 安全保证要求：提出了评估保证级（EAL）

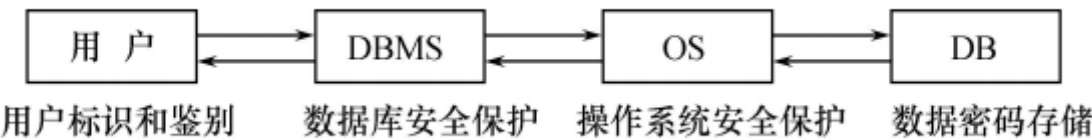
表4.2 CC评估保证级（EAL）划分

评估保证级	定 义	TCSEC安全级别 （近似相当）
EAL1	功能测试（functionally tested）	
EAL2	结构测试（structurally tested）	C1
EAL3	系统地测试和检查（methodically tested and checked）	C2
EAL4	系统地设计、测试和复查（methodically designed, tested, and reviewed）	B1
EAL5	半形式化设计和测试（semiformally designed and tested）	B2
EAL6	半形式化验证的设计和测试（semiformally verified design and tested）	B3
EAL7	形式化验证的设计和测试（formally verified design and tested）	A1

1.3 中国计算机安全标准制定

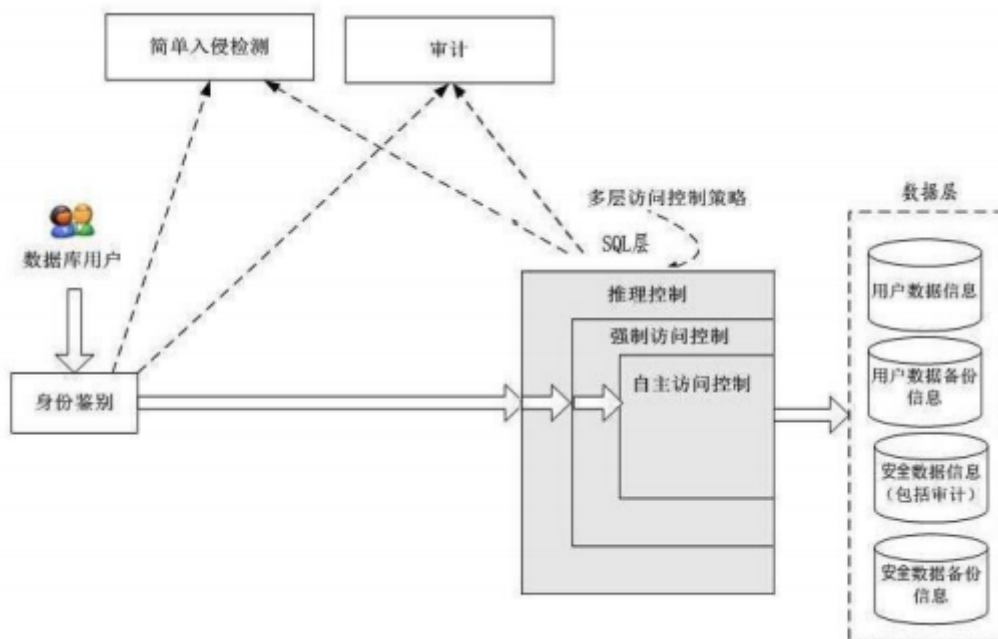
2.数据库安全性控制

2.1 计算机系统的安全模型



层级设置

2.2 数据库管理系统安全性控制模型



数据库安全性控制的常用方法：

- 用户标识和鉴定
- 存取控制
- 视图
- 审计
- 数据加密

2.3 用户身份鉴别（最外层）

用户标识：用户名+用户标识号（唯一）

鉴别方法：

- 静态口令鉴别：用户自己设定
- 动态口令鉴别：每次鉴别时使用动态产生的新口令
- 生物特征鉴别
- 智能卡鉴别：不可复制的硬件，硬件加密

2.4 存取控制

2.4.1 存取控制机制组成

- 定义用户权限，并登记到数据字典中
用户对某一数据对象的操作权力称为权限，安全规则或授权规则
- 合法权限检查

2.4.2 存取控制方法

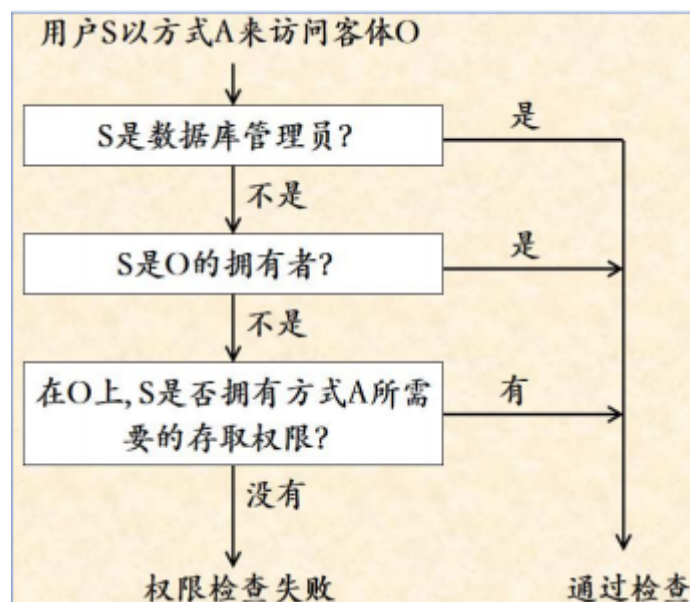
表4.3 关系数据库系统中的存取控制对象与权限

对象类型	对象	操作类型
数据库 模式	模式	CREATE SCHEMA
	基本表	CREATE TABLE, ALTER TABLE
	视图	CREATE VIEW
	索引	CREATE INDEX
数据	基本表和视图	SELECT, INSERT, UPDATE, DELETE, REFERENCES, ALL PRIVILEGES
	属性列	SELECT, INSERT, UPDATE, REFERENCES, ALL PRIVILEGES

- **自主存取控制 (DAC) :C2级**
用户对不同的数据对象有不同的存取权限；不同的用户对同一对象也有不同的权限；用户还可以转授权限。
- **强制存取控制 (MAC) :B1级**
每一个数据对象被标以一定的密级；每一个用户也被授予某一个级别的许可证；对于任意一个对象，只有具有合法许可证的用户才可以存取。用户不能直接感知或进行控制。适用于对数据有严格而固定密级分类的部门：军事，政府。

2.4.3 DAC

- 使用SQL中的GRANT语句和REVOKE语句实现
- 用户权限的组成要素：
 - 数据库对象
 - 操作类型
- 定义用户的存取权限称为授权管理
- 用户存取权限的定义（用户获取在一个客体上的存取权限方法）：
 - 一个客体的属主（创建者）自动拥有该客体上的所有操作权限
 - 拥有权限的用户可以自主地将他拥有的权限传授给其他仁义在数据库系统中的用户
- 访问控制：
 - 在用户登录时进行用户的身份鉴别
 - 在用户访问数据库时执行访问控制检查



- 缺点：可能存在数据的无意泄露；因为数据本身没有安全标记，只是对数据的存取权限进行安全控制。比如甲将某些数据存取权限授权给乙，甲的意图是仅允许乙操作，但是乙可以将数据备份得到副本进行传播。

2.4.4 授权：授予与收回

SQL中的授权机制

- 数据库管理员：拥有所有对象的所有权限；根据情况不同的权限授予不同的用户
- 用户：拥有自己建立的对象的全部的操作权限；可以使用GRANT，把权限授予其他用户
- 被授予的用户：如果具有继续授予的许可，可以把获得的权限再授予给其他用户
- 所有授予出去的权力在必要时又要用REVOKE语句收回

2.4.5 GRANT语句

格式：

```

1 GRANT <权限> [, <权限>] ...
2 ON <对象类型> <对象名> [, <对象类型> <对象名>]...
3 TO <用户> [, <用户>] ...
4 [WITH GRANT OPTION];
  
```

语义：将对指定操作对象的指定操作权限授予指定的用户

- 授权用户（发出或执行GRANT语句的用户）
 - 数据库管理员
 - 数据库对象的创建者（即数据库对象的属主Owner）
 - 拥有该权限的用户
- 被授权用户（接受权限的用户）
 - 一个或多个具体用户
 - 一个或多个用户组（角色ROLE）
 - PUBLIC（即全体用户）

WITH GRANT OPTION 子句:在 GRANT 命令中，如果使用了WITH GRANT OPTION选项，那么被授权用户可以将接受到的权限再授予其他用户，否则，被授权用户不能把接受到的权限再转授给其他用户。

不允许循环授权。

示例：

❑ [例4. 3] 把对表SC的查询权限授予所有用户。

```
GRANT SELECT
ON TABLE SC
TO PUBLIC;
```

❑ [例4. 4] 把查询Student表和修改学生学号的权限授给用户U4。

```
GRANT UPDATE(Sno), SELECT
ON TABLE Student
TO U4;
```

➤ 对属性列的授权，必须明确指出相应属性列名

2.4.6 REVOKE语句

授予的权限可以由数据库管理员或其他授权者用REVOKE语句收回

格式：

```
1 REVOKE <权限> [, <权限>] ...
2 ON <对象类型> <对象名> [, <对象类型> <对象名>]...
3 FROM <用户> [, <用户>] ... [CASCADE | RESTRICT];
```

示例：

❑ [例4. 8] 把用户U4修改学生学号的权限收回。

```
REVOKE UPDATE(Sno)
ON TABLE Student
FROM U4;
```

❑ [例4. 9] 收回所有用户对表SC的查询权限。

```
REVOKE SELECT
ON TABLE SC
FROM PUBLIC;
```

❑ [例4. 10] 把用户U5对SC表的INSERT权限收回。

```
REVOKE INSERT
ON TABLE SC
FROM U5 CASCADE ;
```

➤ 将用户U5的INSERT权限收回的时候使用CASCADE，则同时收回U6或U7的INSERT权限，否则拒绝执行该语句。

➤ 如果U6或U7还从其他用户处获得对SC表的INSERT权限，则他们仍具有此权限，系统只收回直接或间接从U5处获得的权限。

2.4.7 创建数据库模式的权限

由DBA创建

CREATE USER语句格式：

```
1 CREATE USER <username> [WITH] [DBA | RESOURCE | CONNECT]
```

该语句不是SQL标准；只有系统的超级用户才有权限创建一个新的数据库用户。

三种权限：

- CONNECT：default；此类用户不能创建新用户，不能创建模式，也不能创建基本表，只能登录数据库
- RESOURCE：此类用户能创建基本表和视图，成为所创建对象的属主；不能创建模式，不能创建新的用户
- DBA：超级用户，可以创建新的用户、创建模式、基本表、视图；DBA拥有对所有数据库对象的存取权限，还可以授权

拥有的权限	可否执行的操作			
	CREATE USER	CREATE SCHEMA	CREATE TABLE	登录数据库，执行数据查询和操纵
DBA	可以	可以	可以	可以
RESOURCE	不可以	不可以	可以	可以
CONNECT	不可以	不可以	不可以	可以，但必须拥有相应权限

2.5 数据库角色

数据库角色：被命名的一组与数据库操作相关的权限，**角色是权限的集合**。可以为一组具有相同权限的用户创建一个角色，使用角色来管理数据库权限可以简化授权的过程。

角色的创建：

```
1 CREATE ROLE <角色名>
```

给角色授权：

```
1 GRANT <权限> [, <权限>] ...
2 ON <对象类型> <对象名>
3 TO <角色> [, <角色>] ...
```

将一个角色授予其他的角色或用户：

```
1 GRANT <角色1> [, <角色2>] ...
2 TO <角色3> [, <用户1>] ...
3 [ WITH ADMIN OPTION ] ;
```

语义与GRANT语句基本相同。

角色权限的收回：

```
1 REVOKE <权限> [, <权限>] ...
2 ON <对象类型> <对象名>
3 FROM <角色> [, <角色>] ...
```

WITH GRANT OPTION 与 WITH ADMIN OPTION 的区别：

ADMIN OPTION是关于**系统权限**的授予权（系统权限：特定的数据库访问操作，通常是数据定义或数据控制命令，如数据库对象的创建等）。

GRANT OPTION是关于**对象权限**的授予权（对象权限：指数据库对象的访问权限，如表中的数据访问、存储过程的调用等）。

一个角色的权限：**被直接授予这个角色的全部权限，加上通过其他角色授予这个角色的全部权限。**

如果使用CASCADE执行权限的回收操作时，**系统权限不会被级联回收，而对象权限则需要级联回收。**

示例：

□ **[例4.11] 通过角色来实现将一组权限授予一个用户。步骤如下：**

➤ **首先创建一个角色 R1**

CREATE ROLE R1;

➤ **然后使用GRANT语句，使角色R1拥有Student表的SELECT、UPDATE、INSERT权限**

**GRANT SELECT, UPDATE, INSERT
ON TABLE Student
TO R1;**

➤ **将这个角色授予王平，张明，赵玲。使他们具有角色R1所包含的全部权限**

GRANT R1 TO 王平,张明,赵玲;

➤ **可以一次性通过R1来回收王平的这3个权限**
REVOKE R1 FROM 王平;

2.6 强制存取控制 (MAC)

2.6.1 实体

DBMS所管理的全部实体分为主体和客体

- 主体是系统中的活动实体：DBMS所管理的实际用户，也包括代表用户的各进程
- 客体是系统中的被动实体，受主体操纵：文件、基本表、索引、视图

2.6.2 敏感度标记 (Label)

对于实体，DBMS为它们每个实例指派一个Label

敏感度标记级别：

- 绝密 TS
- 机密 S
- 可信 C
- 公开 P

主体的敏感度标记称为许可证级别

客体的敏感度标记称为密级

2.6.3 强制存取控制规则

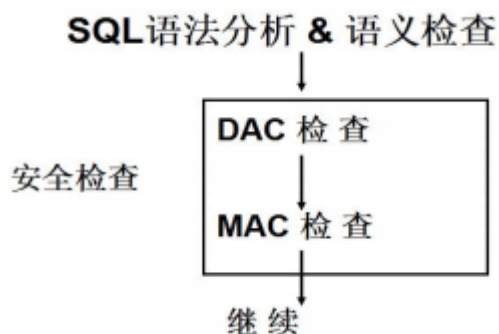
(下读) 仅当主体的许可证级别大于或等于客体的密级时, 该主体才能读取相应的客体

(上写) 仅当主体的许可证级别小于或等于客体的密级时, 该主体才能写相应的客体

强制存取控制 (MAC) 是对数据本身进行密级标记, 无论数据如何复制, 标记与数据是一个不可分的整体, 只有符合密级标记要求的用户才可以操纵数据。

2.6.4 DAC+MAC

实现MAC时要首先实现DAC; 因为较高安全性级别提供的安全保护要包含较低级别的所有保护。



3.视图机制

把要保密的数据对无权存取这些数据用户隐藏起来, 对数据提供一定程度的安全保护。

视图机制间接地实现支持存取谓词的用户权限定义。

示例:

❑ [例4.14] 建立计算机系学生的视图, 把对该视图的SELECT权限授予王平, 把该视图上的所有操作权限授予张明。

❑ 先建立计算机系学生的视图

CS_Student

```
CREATE VIEW CS_Student
AS
SELECT *
FROM Student
WHERE Sdept = 'CS';
```

❑ 在视图上进一步定义存取权限

```
GRANT SELECT
ON CS_Student
TO 王平;
```

```
GRANT ALL PRIVILIGES
ON CS_Student
TO 张明;
```

4.审计

审计设置以及审计日志一般都存储在数据字典中。必须把审计开关打开 (即把系统参数 `audit_trail` 设为 `true`), 才可以在系统表 `SYS_AUDITTRAIL` 中查看到审计信息。

数据库安全审计系统提供了一种事后检查的安全机制。

4.1 概念

审计：

- 启用一个专用的审计日志 Audit Log 将用户对数据库的所有操作记录在上面
- C2以上安全级别的DBMS必须具有审计功能

审计功能的可选性：

- 审计很浪费时间和空间
- 审计主要用于对安全性要求较高的部门
- DBA可以根据应用对安全性的要求，灵活地打开或关闭审计

4.2 审计事件

- 服务器事件：审计数据库服务器发生的事件，包括数据库服务器的启停、配置文件的重新加载等
- 系统权限：对系统拥有的结构或模式对象进行操作的审计；执行相关操作需要的权限是通过系统权限获得的
- 语句事件：对SQL语句的审计，如对DDL、DML、DQL及DCL语句的审计
- 模式对象事件：对特定模式对象上进行的SELECT或DML操作的审计；模式对象主要包括表、视图、存储过程/函数，但不包括表上的索引、约束、触发器等

4.3 审计功能

□基本功能：提供多种审计查阅方式

□多套审计规则：一般在数据库初始化时设定

□提供审计分析和报表功能

□审计日志管理功能

- 防止审计员误删审计记录，审计日志必须先转储后删除；
- 对转储的审计记录文件提供完整性和保密性保护；
- 只允许审计员查阅和转储审计记录，不允许任何用户新增和修改审计记录等。

□提供查询审计设置及审计记录信息的专门视图

4.4 审计级别

- 用户级审计：任何用户可设置的审计；主要是用户针对自己创建的数据库表和视图进行审计
- 系统级审计：只能由数据库管理员设置

4.5 审计语句

❑ **AUDIT**语句 和 **NOAUDIT**语句

- **AUDIT**语句：设置审计功能
- **NOAUDIT**语句：取消审计功能

❑ **[例4. 15]** 对修改SC表结构或修改SC表数据的操作进行审计 **AUDIT ALTER, UPDATE ON SC;**

❑ **[例4. 16]** 取消对SC表上的ALTER和UPDATE操作的审计 **NOAUDIT ALTER, UPDATE ON SC;**

5.数据加密

基本思想：根据一定的算法将原始数据明文转换成密文。

加密方法：存储加密；传输加密

5.1 存储加密

- 透明存储加密：对用户完全透明；内核级加密保护；将数据在写到磁盘时对数据加密，授权用户读取数据时解密
- 非透明存储加密：通过多个加密函数实现

5.2 传输加密

- 链路加密：链路层；报文和报头都加密
- 端到端加密：报文加密；报头不加密

6.其他安全性保护

- 推理控制：避免用户利用能够访问的数据推知更高密级的数据
- 隐蔽信道：间接传输数据
- 数据隐私保护